

AltoMare — 24-Hour Rebuild Roadmap

From SIH PS1288 Prototype to a Demo-Ready, ML-Powered NRW Platform

Team Larpers | New Hackathon Submission | New GitHub Repository | Time Budget: 24 Hours

This document is a single source of truth for the rebuild: it audits the existing **ALTOMARE-clean** codebase against the original Solution Document (PS1288), lists every gap between what was promised and what was actually shipped, integrates the 9 proposed new AI/agent features and two scikit-learn models, flags every old formula or mock path that should be replaced or removed, defines the new repository structure, and lays out an hour-by-hour build plan across the team for the next 24 hours.

How to use this document: Section 1 is the audit (what's real, what's fake, what's missing). Section 2 maps every gap to the original doc. Section 3 is the full new-feature + ML integration plan. Section 4 lists what to deprecate. Section 5 is the new repo layout. Section 6 is the hour-by-hour build order with owners. Section 7 is demo-day risk mitigation and the pitch script.

Part I — Original Solution Document (PS1288)

Reproduced in full below so this single PDF is a complete, standalone brief — no other file needs to be supplied to build from this document.

1. The Problem Statement, Exactly As Given

PS1288, issued by the Ministry of Jal Shakti, asks for the "Use of Digital Technology in addressing Non-Revenue Water (NRW)." The official description: "There is a need to develop a system or technology to trace and tackle non-revenue water and convert it into revenue water using digital methods. This will save water as well as increase profitability and improve the return on investment with respect to water distribution networks."

Three verbs matter, and they are not the same task:

- **Trace** — find and measure where water is being lost, with enough precision to act on it.
- **Tackle** — actually reduce the loss, not just report it.
- **Convert into revenue** — turn that reduction into money the utility actually recovers. This is the part the PS cares about most, because it defines success as profitability and ROI, not just detection.

A solution that only detects physical pipe leaks answers about a third of this problem statement. AltoMare is built to answer all three parts.

2. What Non-Revenue Water Actually Is

Non-Revenue Water is the gap between how much water a utility puts into its distribution network and how much water it actually gets paid for. In India, this gap is close to 38% of all water treated and supplied — more than double the 16% average loss rate in the United States. NRW is officially made up of two very different kinds of loss:

Real losses (physical losses) — Water that physically escapes the system: pipe leaks, bursts, overflow from tanks. A plumbing problem; needs to be found and repaired.

Apparent losses (commercial losses) — Water that is delivered but never billed: meter tampering, illegal or unauthorized connections, under-registering old meters, billing/data-entry errors. Not a plumbing problem at all — a revenue-collection problem, and it often makes up a large share of a utility's total NRW.

Most existing solutions focus almost entirely on real losses. But the PS explicitly asks for water to be "converted into revenue," which is fundamentally about the apparent-loss side too. AltoMare addresses both, using the same underlying data.

3. Why Existing Solutions Don't Fit Most of India

Every serious NRW technology studied — the US DOE's comparison of nine leak-detection methods, satellite systems like ASTERRA, digital-twin tools like KROHNE's ERT™, Indian players like SmartTerra — assumes the town already has one of three things: money for expensive equipment, an existing SCADA/smart-metering system, or a trained technical team to run it.

That assumption does not hold for most of India. Only about 8–9% of AMRUT 2.0 towns have even been approved for SCADA monitoring as of early 2026, and under the Jal Jeevan Mission, sensor-based monitoring pilots have reached fewer than 0.2% of the six-lakh-plus villages covered. The overwhelming majority of Indian towns and almost all rural JJM schemes run on manual billing registers, paper meter readings, and periodic lab tests — nothing digital yet.

This is the gap AltoMare is built for: a system that works with whatever data a town already has today, and gets more precise as a town adds instrumentation, instead of requiring it upfront.

4. The AltoMare Data Model: Three Levels, Not One-Size-Fits-All

Instead of assuming every town has the same digital infrastructure, AltoMare defines three data levels. A town is assessed once and placed at the level matching what it already has — no town excluded for lacking equipment, and every town can move up as budget allows.

Level 0 — Manual: No sensors at all. Just billing registers, a bulk meter at the tank/pump outlet if one exists, and periodic lab test results. Data reaches AltoMare via a simple web form or CSV upload; weekly lab results entered by hand.

Level 1 — Partial instrumentation: A handful of low-cost pressure or flow sensors at a few key points, not the whole town. Low-cost microcontrollers (ESP8266/ESP32) sending readings over MQTT/HTTP.

Level 2 — Fully instrumented: Existing SCADA system or smart meters already deployed. Direct API connection to the utility's existing system — no new hardware needed.

This document focuses mainly on Level 0 and Level 1, because that is where over 90% of India's water infrastructure actually sits today.

5. Level 0 — Precisely Measuring NRW With Zero Sensors

The basic water balance: $\text{Loss}(\text{zone}) = \text{Inflow}(\text{zone}) - [\text{Billed Consumption}(\text{zone}) + \text{Authorized Unbilled Use}(\text{zone})]$.

On its own, this single number is too blunt to act on. AltoMare improves on this at Level 0 using four techniques that need no new hardware:

5.1 Component-based water audit (IWA/AWWA standard). Using data most towns already hold on paper — length of pipe mains, number of connections, typical system pressure — AltoMare calculates the Unavoidable Annual Real Losses (UARL): the background leakage every system has even when perfectly maintained. Anything above this baseline is the real, actionable target. This turns "your town loses 38% of its water" into "of that 38%, this much is unavoidable and this much is genuinely fixable."

5.2 Minimum Night Flow (MNF) estimation from pump run-hours. Between roughly 2–4 AM, legitimate household use is close to zero almost everywhere. Any water still being pumped during this window is very likely being lost. Estimated night-time loss $\approx$ Pump run-hours (2–4 AM) $\times$ Rated pump discharge. Most pump operators already log ON/OFF times by hand, or these can be pulled from electricity meter run-hours — no flow sensor required.

5.3 Billing anomaly scoring. Targets apparent loss. AltoMare compares each household's billed consumption against a simple benchmark built from household size or property type, using data ULBs typically already hold in property tax or ration card records. A household billed far less than similar nearby households is flagged as a candidate for meter tampering, an unauthorized connection, or a billing error — sent to the billing department for audit, not a repair crew.

5.4 Free satellite soil-moisture check (Sentinel-2 NDWI). As a low-cost second opinion on a flagged zone, AltoMare can pull freely available Sentinel-2 imagery and check the Normalized Difference Water Index (NDWI) along known pipe routes. Unusual soil moisture or vegetation greening near a pipeline adds independent evidence to a flagged zone. This only catches large, sustained leaks at 10-metre resolution — it will not find a small leak — but it is genuinely free and adds a second, independent signal without new equipment.

6. What Level 0 Honestly Cannot Do

At Level 0, AltoMare can tell a utility which zone is losing water and roughly how much — it cannot pinpoint a leak's exact location within that zone. Precise location requires comparing readings from at least two points, and Level 0 has no sensors to compare.

This is not a hidden weakness. Two things make it acceptable and still valuable: today, the baseline in almost every Level 0 town is zero visibility — no one knows a zone is losing water at all until a pump runs dry or a citizen complains. Going from nothing to "these 2 zones out of 12 need attention this month, ranked by estimated loss" is a large jump in usefulness even without a street address. Within a flagged zone, AltoMare can still narrow the search using data that costs nothing extra — cross-referencing pipe age/material records (older cast-iron sections fail more often) and layering in citizen-reported complaints (low pressure, visible

wetness) through the same WhatsApp/SMS channel used for alerts.

Precise, sensor-based location becomes possible once a town adds even minimal Level 1 instrumentation — an incremental, low-cost next step, not a full system replacement.

7. Level 1 — Converting NRW Into Revenue Water

Once a town has even minimal Level 1 instrumentation — two or three pressure points and one flow meter at a zone boundary — AltoMare adds three features aimed directly at the PS's core ask: turning traced loss into recovered money.

7.1 Automatic meter and theft audit list. When a zone shows high water loss but no matching pressure drop or leak signal, that is a strong sign the loss is apparent, not physical — tampering, an illegal connection, or a billing error. AltoMare automatically generates a ranked audit list for the billing department, sorted by a suspicion score built from consumption drop, meter age, and complaint history.

7.2 Revenue Recovered tracker. When a flagged zone gets a repair or billing correction, AltoMare re-measures that zone's water balance over the following 2–3 billing cycles and calculates actual money recovered: $\text{Revenue Recovered} = (\text{Litres recovered after fix}) \times (\text{Local water tariff rate})$. This accumulates on the dashboard as a running total — e.g. "■X recovered this quarter" — directly matching the PS's own definition of success.

7.3 Repair prioritization by return on investment. With several zones flagged at once, a budget-limited utility needs to know which to fix first. AltoMare estimates each leak's daily revenue loss (litres lost $\times$ tariff rate) against a rough repair cost estimate based on pipe type/age, and calculates a simple payback period: $\text{Payback period} \approx \text{Estimated repair cost} \div \text{Daily revenue loss}$ from that zone. A zone that pays for its own repair in three weeks should be fixed before one that takes eight months.

All three are software-only — no new sensors required, only better logic layered onto data the system already collects.

8. A Secondary Layer: Water Quality and Contamination Risk

Leaking pipes do not just waste water — the pressure drop can pull surrounding groundwater back into the pipe, contaminating the water people are about to drink. AltoMare includes a second engine, Latios, which tracks six basic water-safety indicators (pH, turbidity, TDS, chlorine, bacteria, hardness) per zone, and a correlation layer checking whether a leak and a contamination signal are showing up in the same zone around the same time.

This is a secondary, additive layer, not the core answer to PS1288 — the PS is specifically about non-revenue water and its financial recovery, not water safety. Latios adds real public-health value on top of the core NRW solution and is a genuine differentiator, but should not be mistaken for the primary response to what this problem statement is actually asking for.

8.1 How Latios works at Level 0, honestly. At true Level 0, most of Latios's six parameters cannot be continuous. The realistic version relies on manual field-test kits (strip/titration kits for pH and chlorine) and periodic lab results for bacteria, entered through the same web form used for billing data — this piggybacks on testing already mandated under JJM's Field Test Kit (FTK) guidelines. Two honest consequences: Level 0 Latios is not real-time, and the correlation engine's precision drops — matching a leak to a quality reading within a tight time window isn't possible when both arrive on slow, independent schedules. At Level 0 correlation becomes a coarser check (same reporting period) rather than a near-real-time match.

9. How AltoMare Directly Answers PS1288

Trace NRW — Component-based water audit, Minimum Night Flow estimation, and billing anomaly scoring, all working from Level 0, no sensors required.

Tackle NRW — Zone-level leak flagging (Latias) plus pipe-age and citizen-report narrowing within a flagged zone; sharper, sensor-based location as a town adds Level 1 instrumentation.

Convert to revenue water — Automatic meter/theft audit lists, the Revenue Recovered tracker, and ROI-based repair prioritization — the direct, software-only answer to the PS's core financial ask.

Increase profitability / improve ROI — Payback-period calculation per leak, and a running recovered-revenue total on the dashboard — a concrete, ongoing measure of financial impact, not just a one-time detection report.

10. Technology Summary

Backend: Python (FastAPI) — serves both Latias and Latios, supports real-time alerts.

Database: PostgreSQL with PostGIS — stores zones, readings, billing data, and alerts; PostGIS handles zone boundary mapping.

Analytics: pandas, NumPy, and rule-based logic for water balance, MNF, and billing-anomaly scoring; scikit-learn as a future upgrade once historical data accumulates.

Level 1 hardware (optional): ESP8266/ESP32 microcontrollers with pressure and flow sensors, over MQTT/HTTP.

Frontend: React with Recharts (trend charts) and Leaflet (zone map).

Alerts: WhatsApp Business API and Twilio SMS — reaching field staff directly, no app required.

Satellite layer: Free Sentinel-2 imagery via public APIs for the NDWI soil-moisture check.

11. Honest Limitations and Roadmap (Original)

Stated directly rather than left to be discovered under questioning:

- Level 0 gives zone-level detection, not street-level location. Precise location needs at least minimal Level 1 instrumentation.
- Billing-data quality directly affects accuracy at Level 0 — the water balance calculation is only as good as the billing records feeding it; NRW data in India is often incomplete or inconsistent.
- Bacteria testing remains a weekly lab process even at Level 1, so the fastest-moving part of Latios's contamination detection has an inherent reporting lag; other parameters (pH, turbidity) can be near real-time only where sensors exist.
- The correlation between a leak and a contamination reading in the same zone/time window is a strong statistical signal, not proof of causation — a priority flag for field verification, not an automatic conclusion.

The original roadmap addresses each directly: Level 1 rollout for location precision, data-quality validation checks on incoming billing records, and expanding lab-test partnerships to shorten the bacteria reporting cycle over time.

12. Conclusion (Original Solution Document)

AltoMare's core contribution to PS1288 is treating non-revenue water as what it actually is — partly a plumbing problem and partly a revenue-collection problem — and answering both halves with data most Indian towns already have, starting from zero sensors. Leak detection alone would have answered only part of what the Ministry of Jal Shakti is asking for. By adding apparent-loss detection, the Revenue Recovered tracker, and ROI-based repair prioritization, AltoMare is built to trace, tackle, and convert non-revenue water into revenue water — exactly as the problem statement asks — while remaining deployable in the towns and rural JJM schemes that make up the overwhelming majority of India's water infrastructure today.

Part II — Rebuild Audit & 24-Hour Roadmap

Everything from here on assesses the actual ALTOMARE-clean codebase against Part I above, and lays out the complete plan to close every gap within a 24-hour build window.

1. Codebase Audit — What's Actually Real Right Now

Verified directly from ALTOMARE-clean.zip (backend: FastAPI + SQLAlchemy + Postgres/Supabase; frontend: React + TS + Tailwind + Leaflet). This is the ground truth the roadmap builds from.

1.1 Backend — endpoint-by-endpoint reality check

Endpoint	Status	Reality
GET /zones	LIVE	Reads real zones table
POST /readings	LIVE	Writes raw_readings (inflow/pressure/flow)
GET /alerts	LIVE	Reads leak_alerts table, filters by zone/status
POST /detect/{zone_id}	LIVE	Triggers water_balance detection only — MNF, PPA, billing-anomaly are never called from any endpoint, even though the functions exist
POST /quality	LIVE	Writes quality_readings; auto-triggers correlation check
GET /correlation/{zone_id}	LIVE	Reads correlation_events
GET /nrw/summary	LIVE	Real NRW% = (Inflow-Billed)/Inflow per zone — good, not in original 7-endpoint contract
GET /revenue/summary	LIVE	Reads revenue_log — but revenue_log is only ever populated by manually calling /revenue/recover; nothing auto-creates entries when a leak is marked resolved
POST /revenue/recover	LIVE	Manual-trigger only, no UI, not in Swagger-tested flow
POST /alerts/notify	LIVE + fallback	Twilio real if configured; safe mock fallback otherwise — this pattern is good and should be reused for the LLM calls
GET /audit/billing/{zone_id}	LIVE but INVISIBLE	Real ranked theft/tampering list — zero frontend page renders it
GET /audit/ppa/{zone_id}	LIVE but INVISIBLE, and SIMULATED	Leak-location narrowing works but pressure readings are synthetically generated (simulate_ppa), not real Level-1 sensor data — this is honest per the doc (Level 1 hardware out of scope) but must be labelled 'simulated' everywhere it's shown, including in any new UI

1.2 Backend logic that exists but is disconnected or fake

Item	Location	Why it matters
confidence_score hardcoded to 0.7	engines/latias.py, process_zone()	Every alert written by the only active detection path (water_balance) gets the exact same fake confidence number, regardless of data quality. This is the single most damaging thing a technical judge will catch.
mnf_loss_estimate()	engines/latias.py	Fully implemented, unit-tested in __main__, never called from any router. 'Multi-technique detection' is currently one technique in production.
unavoidable_annual_real_losses() (UARL)	engines/latias.py	Implemented, never called or exposed. Section 5.1 of the solution doc (component-based water audit) is not actually running anywhere.
flag_anomalous_households()	engines/latias.py	Real rule-based scoring (billed vs benchmark, 40% threshold) — works, but purely rule-based, no ML, and only reachable via the invisible /audit/billing endpoint.

Item	Location	Why it matters
calculate_payback_period()	engines/revenue.py	Function exists and is correct. No endpoint exposes it anywhere — cannot even be tested in Swagger.
TARIFF_RATE_PER_LITRE = 0.005	engines/revenue.py	Hardcoded module-level constant, same rate for every town/zone, no config or per-town override.
No authentication anywhere	main.py / all routers	Every endpoint, including /revenue/recover and /alerts/notify, is open with no auth layer.
Tests	backend/test/	Only test_latias.py exists — latios.py (quality+correlation), revenue.py, and every router are completely untested.

1.3 Frontend — mock data currently masquerading as live

Feature	Location	Reality
NRW Trend chart	data/dashboardDemoData.ts → demoNRWTrend, consumed via services/dashboardData.ts useNRWTrend()	100% hardcoded 8-point array with an artificial setTimeout delay to fake a network call. Never touches the backend at all.
Quality Trend chart	dashboardDemoData.ts → demoQualityTrend, useQualityTrend()	Same pattern — fully fake, fake loading delay included.
Zone map silent fallback	services/api.ts → getZones()	If GET /zones throws, the catch block silently returns mockZones (10 Delhi-coordinate fake zones) with no error banner or 'showing demo data' indicator — a judge would see a populated map with zero signal it isn't real.
Settings page	pages/Settings.tsx	To its credit, already self-labels 'NRW Trend: DEMO' and 'Water Quality: DEMO' with a 'demo' badge — this honesty pattern should be extended everywhere else mock data still exists, not removed.

1.4 The core problem, restated

The most differentiated, most PS1288-aligned work — billing-fraud detection, leak-location narrowing, component-based audit, payback-period ROI — is real, working, and completely invisible in the product. Meanwhile the most visible screens (the two trend charts, and the zone map's failure state) are fake. That is exactly backwards for a judged demo, and fixing that inversion is worth more than any single new feature in this roadmap.

2. Gap Map — Solution Document vs. What Was Built

Every claim in AltoMare_Solution_Document_PS1288.pdf, checked against the actual repo.

Solution-doc section	What was promised	What exists in repo	Verdict
§5.1 Component-based water audit (UARL)	Documented as core Level-0 technique	Function written, never called or exposed	MISSING (wire it in)
§5.2 Minimum Night Flow (MNF)	Documented as core Level-0 technique	Function written, never called from process_zone() or any endpoint	MISSING (wire it in)
§5.3 Billing anomaly scoring	Documented as core apparent-loss technique	Real, working, but rule-based only and hidden (no frontend)	PARTIAL — needs ML upgrade + UI
§5.4 Sentinel-2 NDWI satellite check	Documented as free second-opinion signal	Zero code anywhere in the repo	NOT BUILT
§4 CSV/bulk upload (Level 0 data entry)	“Simple web form or CSV upload”	Only single-row POST /readings and POST /quality exist; no bulk ingestion path	NOT BUILT
§6 Leak location narrowing (PPA)	Documented, explicitly simulated at Level 0/1 by design	Built as simulate_ppa(), correctly labelled internally as simulated — but invisible on frontend	PARTIAL — needs UI
§7.1 Meter/theft audit list	Core revenue-conversion feature	Real, working, invisible	PARTIAL — needs UI
§7.2 Revenue Recovered tracker	Core revenue-conversion feature	Formula correct; only populated by a manual, unauthenticated POST with no automatic trigger when an alert is resolved	PARTIAL — needs automation
§7.3 Payback-period ROI ranking	Core revenue-conversion feature	Formula correct, zero endpoint, not testable, not on any dashboard	MISSING (wire it in)
§8 Latios — 6-parameter water quality engine	Secondary layer	Fully real: thresholds, SAFE/WARNING/ALERT/CRITICAL classification, correlation with time-window logic by data level	DONE — best-built part of the repo
§9 PS1288 answer table (trace/tackle/convert)	The entire pitch narrative	Trace: partially real (water balance only). Tackle: simulated PPA, invisible. Convert: formulas real, zero automation, zero UI	Story is not yet demonstrable end-to-end
§10 Tech stack — scikit-learn	Listed as “future upgrade once historical data accumulates”	Zero ML anywhere; everything is deterministic rules/formulas	NOT BUILT — this hackathon is exactly the trigger point the doc describes

Reading this table straight: nothing in the original doc was faked in the sense of lying about results — every number the system produces is a real calculation. The gap is entirely completion and visibility: half the promised techniques are unwired, and the differentiated half of the product has no UI at all.

3. New Feature + ML Integration Plan

Combines the 9-feature list from the team brainstorm with the two scikit-learn models, tiered by build cost vs. demo impact, and mapped onto the real engines above (not built as parallel/duplicate logic).

3.1 Tier 0 — Foundation (build first; everything else depends on this)

Feature	Tech	Why / how it connects
One-Town Data Pipeline (CSV upload)	Python + pandas + FastAPI UploadFile	Solves §4's never-built CSV path AND is the mechanism to load Lucknow's real published aggregate (55% NRW, █650M/yr) plus zone-level synthetic data that sums consistently to that real number. Every other feature is more credible once it's running on this instead of the current ad-hoc <code>synthetic_data_generator.py</code> .
Wire MNF + UARL into <code>process_zone()</code>	Existing functions, already written	<code>process_zone()</code> currently only calls <code>water_balance_loss()</code> . Extend it to also run <code>mnf_loss_estimate()</code> and <code>unavoidable_annual_real_losses()</code> and store the method actually used. Turns 'one technique' into the three-technique story the doc promises — near-zero new code.
Real confidence score (replace hardcoded 0.7)	scikit-learn OR weighted formula	Compute from data completeness (are inflow + billing + MNF all present?), reading count, and variance instead of a constant. See 3.3 for the ML version — this single fix answers the sharpest question any judge will ask.

3.2 Tier 1 — The Differentiator Cluster (AI Copilot)

Grouped as one feature, not four, per the original evaluation — the agent, evidence engine, recommendations, and UI only matter together.

Feature	Tech	Notes
AI Investigation Agent	Python + FastAPI + Groq LLM + tool/function calling	Wraps the existing engines as callable tools — not new detection logic, a reasoning layer over what already works: <code>run_water_balance</code> , <code>run_mnf</code> , <code>run_uarl</code> , <code>run_billing_anomaly</code> (upgraded to Isolation Forest), <code>run_ppa</code> , <code>get_correlation_status</code> .
Evidence & Explanation Engine	Pydantic + existing DB reads	Every AI claim must cite the actual DB rows behind it (the real inflow/billed numbers, the real anomaly scores) — this is what makes it defensible under judge questioning instead of a black box.
AI Recommendations	Groq LLM + structured JSON output (Pydantic schema)	Given evidence, output a structured <code>{action, urgency, estimated_cost, estimated_payback_days}</code> object — reuses <code>calculate_payback_period()</code> directly rather than having the LLM invent numbers.
Confidence + Missing-Data Handling	Prompt-level rules + Pydantic validation	Agent must say 'insufficient evidence' when a zone is missing inflow or billing data, rather than reasoning over nulls. Cheap, and directly fixes the fake-precision problem from a second angle.
AI Copilot UI	React + TypeScript + Tailwind	Judge clicks a zone → sees the agent investigate live, citing real numbers, ending in a recommendation. This is the actual demo moment — none of the above matters without this screen existing.

Feature	Tech	Notes
Groq fallback pattern	Mirrors alerts/notify.py's Twilio fallback	Every LLM call needs a try/except that degrades to a rule-based summary (e.g. 'Loss = X L, method = water_balance, no AI commentary available') instead of crashing the UI if Groq times out mid-demo.

3.3 Tier 1 — Machine Learning (scikit-learn, both models)

Feature	Approach	Why / integration notes
Model 1: Billing Anomaly Detector	Isolation Forest (sklearn.ensemble.IsolationForest), fallback Local Outlier Factor	Replaces / augments flag_anomalous_households()'s fixed 40%-below-benchmark rule with genuine unsupervised anomaly detection over billed_litres, benchmark_litres, and household metadata. Trains in milliseconds on the synthetic dataset, needs no GPU. This is the real 'we use ML' answer for judges — the algorithm class is the textbook-correct tool for this exact problem (unlabelled outlier detection), not a bolted-on buzzword.
Model 2: Confidence Score Model	Lightweight regression (sklearn.linear_model or a hand-built weighted score with documented weights) over: data completeness (0-1), number of readings, variance across readings, and which detection methods agreed	Directly replaces the hardcoded confidence_score = 0.7. Two features solved with one model: (a) real ML on the dashboard, (b) fixes the single most-flagged weakness in the codebase. Feed this into every write_leak_alert() call site instead of the literal 0.7.
(Optional, time-permitting) Trend forecasting	Simple linear regression over historical NRW snapshots (needs a snapshots table — see Tier 2)	'If unfixed, projected loss over next 30 days is X litres / ■Y' — ties directly into the payback-period/ROI story. Cut first if time runs short; the two models above are non-negotiable, this one is a bonus.

3.4 Tier 2 — Strong, add if time allows, cut first under pressure

Feature	Tech	Notes
Human Approval Workflow	FastAPI + Postgres + React	Officer approves/rejects the agent's recommendation. Pairs with Action/Recovery Tracking for a genuine responsible-AI story: AI recommends → human approves → system tracks whether it worked.
Action / Recovery Tracking	Postgres + existing revenue.py	When an approved action is marked done, auto-call calculate_revenue_recovered() and write to revenue_log — this is also what fixes the 'nothing auto-populates revenue_log' gap found in §1.2/§2.
Persistent Investigation Memory	One new Postgres table (zone_id, past incidents, past actions, outcomes)	'This zone was flagged before, here's what happened' — cheap (one table), adds depth on a second look rather than the first-glance wow factor. Lowest priority in this tier.

4. What to Deprecate, Replace, or Explicitly Retire

Being upfront about this in the repo (a short DEPRECATIONS.md, or comments at the point of change) is itself a credibility signal for judges — it shows the team knows the difference between a prototype shortcut and a finished answer.

Item	Action	Reasoning
confidence_score = 0.7 hardcoded constant	REPLACE	Swap for the Model 2 confidence score (3.3) at every write_leak_alert() call site. Do not leave both paths in the code — the fake constant must be deleted, not just unused.
Rule-based flag_anomalous_households() 40%-threshold logic	REPLACE (keep as fallback)	Isolation Forest becomes the primary path. Keep the old rule-based function only as the documented no-ML fallback if sklearn or the model fails to load — same pattern as the Twilio/Groq fallbacks, not dead code.
demoNRWTrend / demoQualityTrend arrays + useNRWTrend()/useQualityTrend() fake-delay hooks	DELETE	Once GET /nrw/summary has a real time-series (needs the new snapshots table, Tier 2/roadmap hour block below), point the charts at real history. No reason for a fake setTimeout to survive a from-scratch rebuild.
Silent mock-zone fallback in getZones()	REPLACE	Keep resilience (don't crash the map), but replace 'silently return fake Delhi zones' with an explicit visible error state / retry banner. Never show unlabeled fake data as if live again anywhere in the new repo.
Settings page 'DEMO' badges on trend data	KEEP the pattern, remove the need for it	The self-labelling instinct is good; the goal is for there to be nothing left needing that badge except explicitly-simulated PPA data (see next row), which SHOULD keep an honest label.
simulate_ppa() synthetic pressure readings	KEEP, but label clearly everywhere in UI	This is a legitimate, documented design choice (Level 1 hardware genuinely out of scope) — not a shortcut to hide. Every place it's surfaced in the new Copilot UI must say 'Simulated Level-1 sensor data' explicitly.
TARIFF_RATE_PER_LITRE hardcoded module constant	UPGRADE	Move to a per-zone/per-town configurable value sourced from the new data pipeline's town profile, defaulting to the current ■0.005/litre if not provided — not a structural rebuild, just stop hardcoding it in a formula file.
Manual-only /revenue/recover with no trigger	AUTOMATE	Once Action/Recovery Tracking (Tier 2) exists, a resolved+approved alert should call this automatically. Keep the manual endpoint for testing/demo control, but it should no longer be the only path.

5. New Repository Structure

A fresh GitHub repo for the new hackathon, carrying forward the working engines and cutting the mock paths from day one rather than retrofitting.

```
altomare-v2/
■■■■ README.md                (PS1288 framing, honest limitations section, architecture diagram
)
■■■■ DEPRECATIONS.md          (Section 4 of this doc, in the repo)
■■■■ .env.example              (DATABASE_URL, GROQ_API_KEY, TWILIO_*, never real secrets)
■■■■ backend/
■   ■■■■ app/
■   ■   ■■■■ main.py
■   ■   ■■■■ db.py
■   ■   ■■■■ models.py        (Pydantic contracts – add AgentRecommendation, ConfidenceInput,
CSVUploadResult)
■   ■   ■■■■ engines/
■   ■   ■   ■■■■ latias.py      (water_balance, MNF, UARL – now all wired into process_zone)
■   ■   ■   ■■■■ latios.py      (quality + correlation – carried over, already solid)
■   ■   ■   ■■■■ revenue.py     (revenue recovered, payback period – now exposed + automated)
■   ■   ■   ■■■■ ml_billing_anomaly.py (NEW – Isolation Forest model, train + predict)
■   ■   ■   ■■■■ ml_confidence.py (NEW – confidence scoring model)
■   ■   ■   ■■■■ agent.py        (NEW – Groq tool-calling agent, evidence assembly, fallback)
■   ■   ■■■■ routers/
■   ■   ■   ■■■■ core.py         (existing 7+ endpoints, cleaned up)
■   ■   ■   ■■■■ audit.py        (NEW – billing audit + PPA, promoted out of core.py, now with UI
)
■   ■   ■   ■■■■ agent.py        (NEW – POST /agent/investigate/{zone_id})
■   ■   ■   ■■■■ data_pipeline.py (NEW – POST /data/upload-csv, GET /data/town-profile)
■   ■   ■   ■■■■ actions.py      (NEW – approval workflow + recovery tracking)
■   ■   ■■■■ alerts/notify.py    (carried over as-is – fallback pattern already correct)
■   ■■■■ data/
■   ■   ■■■■ db_schema.py        (carried over + new tables: snapshots, action_log, investigation
_memory)
■   ■   ■■■■ lucknow_seed.py      (NEW – replaces synthetic_data_generator.py: real Lucknow aggreg
ate anchor + consistent synthetic zones)
■   ■■■■ test/                   (NEW coverage: test_ml_billing.py, test_confidence.py, test_agenc
y_fallback.py, test_routers.py)
■■■■ frontend/
■   ■■■■ src/
■   ■   ■■■■ pages/
■   ■   ■   ■■■■ Dashboard.tsx   (trend charts now real, not demo arrays)
■   ■   ■   ■■■■ Copilot.tsx     (NEW – the AI investigation screen, the demo centerpiece)
■   ■   ■   ■■■■ BillingAudit.tsx (NEW – surfaces GET /audit/billing, currently invisible)
■   ■   ■   ■■■■ LeakLocation.tsx (NEW – surfaces GET /audit/ppa, labelled 'simulated')
■   ■   ■   ■■■■ DataUpload.tsx  (NEW – CSV upload UI for the data pipeline)
■   ■   ■■■■ services/api.ts     (error states instead of silent mock fallback)
■   ■   ■■■■ data/               (mockData.ts kept ONLY for local dev/storybook, never a runtime
fallback)
■■■■ docs/
■   ■■■■ roadmap.pdf             (this document)
```

6. Team Split — 4 People, Equal Load

Work is divided into four equal ownership lanes so all four people can build in parallel for the full 24 hours with minimal blocking on each other. **Data and ML are kept with the same person** — the ML models train directly on the seed data, so splitting them across two people would mean constant handoff for no benefit.

Person	Lane	Scope
Person 1	Data + ML	Owns everything that touches the dataset end-to-end: the CSV upload pipeline, the Lucknow-anchored seed script, and both scikit-learn models (Isolation Forest for billing anomaly, confidence scoring) — since both models train directly on the data this person is already building.
Person 2	Backend + Detection Engines	Owns the FastAPI/Postgres side of Latias: wiring MNF + UARL into process_zone(), promoting/building the audit endpoints, revenue automation (payback endpoint, auto-populating revenue_log), the approval-workflow backend, and schema/infra for new tables.
Person 3	AI Agent (Copilot backend)	Owns the Groq tool-calling agent end-to-end: wrapping the existing + new engines as tools, the evidence engine, structured JSON recommendations, the insufficient-evidence rule, and the LLM fallback pattern.
Person 4	Frontend + Demo Ops	Owns every screen: killing the two mock trend charts, the zone-map error state, and building BillingAudit.tsx, LeakLocation.tsx, Copilot.tsx, DataUpload.tsx, and the Action Log — plus final deploy and demo rehearsal coordination.

Each hour block below tags every task with the person who owns it. Where a task depends on another person's earlier output (e.g. Frontend charts need Person 1's snapshot data flowing, or the Agent needs Person 2's new endpoints live), that dependency is called out explicitly so nobody is blocked waiting on an implicit handoff.

6.1 Hour-by-Hour Roadmap

Sequenced so the foundation (real data, real confidence, wired formulas) lands before anything AI-flavoured — every later hour is more convincing because earlier hours made the underlying engines real.

Hours 0–2 — Setup + Data Pipeline Foundation

Owner	Tasks
All 4	Create new GitHub repo per Section 5 structure; agree on branch strategy; set up shared .env.example
Person 4	Deploy empty skeleton to Render/Railway (backend) + Vercel (frontend) immediately, not at the end
Person 1	Build POST /data/upload-csv + lucknow_seed.py: real Lucknow aggregate (55% NRW, █650M/yr) anchoring zone-level synthetic data that sums to that figure
Person 2	Port db_schema.py to the new database; add new tables: nrw_snapshots (for real trend charts), action_log, investigation_memory
Person 3	Read through agent.py requirements, set up Groq API access, sketch the tool-calling schema (no build yet — waiting on Person 2's endpoints)

Hours 2–4 — Kill Mock Data + Wire Missing Formulas

Owner	Tasks
Person 4	Delete demoNRWTrend/demoQualityTrend + fake-delay hooks; wire charts to GET /nrw/summary (depends on Person 1's seed data being live); replace silent mock-zone fallback with a visible error state
Person 2	Extend process_zone() to also run mnf_loss_estimate() and unavoidable_annual_real_losses(); store which method(s) actually ran per alert; expose payback period via new GET /revenue/payback/{zone_id}; make TARIFF_RATE configurable per zone
Person 1	Finish seeding households with benchmark data for the billing-anomaly model; verify zone-level sums against the real Lucknow aggregate
Person 3	Draft the agent's tool function signatures against Person 2's real endpoint contracts

Hours 4–7 — Surface the Hidden Differentiators

Owner	Tasks
Person 4	Build BillingAudit.tsx (GET /audit/billing) and LeakLocation.tsx (GET /audit/ppa, clearly labelled 'simulated sensor data')
Person 2	Promote audit endpoints out of core.py into routers/audit.py for clarity; add pagination if household lists are long
Person 1	Start building ml_billing_anomaly.py: Isolation Forest trained on the seeded households dataset
Person 3	Continue agent scaffolding; write the evidence-engine schema (Pydantic) that will cite real DB rows

Hours 7–10 — Machine Learning — Both Models

Owner	Tasks
Person 1	Finish ml_billing_anomaly.py (Isolation Forest, replacing the 40%-rule as primary path, rule kept as documented fallback); build ml_confidence.py: confidence score from data completeness + reading count + variance + method agreement; replace confidence_score=0.7 at every call site — delete the hardcoded constant entirely; unit-test both against the known seeded theft households
Person 2	Wire Person 1's confidence model output into write_leak_alert(); finalize the actions/approval-workflow schema (tables + Pydantic models) so Person 2 can build the router next block
Person 3	Build the agent's core tool-calling loop against Person 2's live endpoints (water balance, MNF, UARL are ready; billing-anomaly tool waits on Person 1's model landing this block)
Person 4	Build DataUpload.tsx CSV upload UI against Person 1's pipeline

Hours 10–15 — AI Copilot Cluster (the demo centerpiece)

Owner	Tasks
Person 3	Complete agent.py: all six tools wired (water balance, MNF, UARL, ML billing-anomaly, PPA, correlation status); structured JSON recommendation output calling calculate_payback_period(); insufficient-evidence rule; try/except fallback to a rule-based summary if Groq errors or times out, mirroring notify.py's fallback pattern
Person 4	Build Copilot.tsx: judge clicks a zone → live investigation trace → evidence → recommendation. This screen is the single highest-leverage thing in the whole 24 hours
Person 2	Build routers/actions.py: officer approve/reject endpoint, ready for Person 4 to wire into the Copilot screen next block

Owner	Tasks
Person 1	Support: tune model thresholds against real demo zones so the agent's citations look sharp; start the optional trend-forecasting model if ahead of schedule

Hours 15–19 — Approval Workflow + Recovery Tracking

Owner	Tasks
Person 2	Finish routers/actions.py: on approval + resolution, auto-call revenue.recover so revenue_log finally populates itself instead of needing a manual POST
Person 4	Add approve/reject buttons to the Copilot screen; build a simple Action Log page showing approved → executed → revenue recovered
Person 3	Harden the agent fallback path — deliberately break the Groq key and confirm the UI degrades gracefully, not crashes
Person 1	If ahead of schedule: Persistent Investigation Memory table + a 'previously flagged' note wired into the zone detail view

Hours 19–22 — Data Story Polish + Stretch Goals

Owner	Tasks
Person 1	Verify every zone's synthetic numbers still sum correctly to the real Lucknow aggregate after all the new writes; add the town-profile display citing the real source; Sentinel-2 NDWI check only if every Tier 0/1 item is done — first thing to cut if behind schedule
Person 2	Bug-fix pass across all new endpoints; confirm every response model matches Pydantic contracts
Person 3	Bug-fix pass on agent edge cases (missing data, zone with no alerts, concurrent requests)
Person 4	Visual polish pass across all new pages; confirm no page shows unlabeled fake data anywhere

Hours 22–24 — Deploy, Rehearse, Harden

Owner	Tasks
Person 4	Final deploy (not localhost); lead the run-through of the demo script (Section 7)
Person 2	Final DB sanity check; confirm the demo dataset resets cleanly if it needs to be rerun
Person 3	Confirm the Groq fallback actually degrades gracefully by testing with a bad API key live, one more time, right before judging
Person 1	Write the honest-limitations README section carried over from the solution doc; confirm data-source citation is accurate
All 4	Click every page once as a judge would: Dashboard → Zones → Copilot → Billing Audit → Leak Location → Revenue — confirm nothing shows unlabeled fake data

7. Demo-Day Risk Mitigation & Pitch Structure

7.1 Risks to design around from hour zero

Risk	Mitigation
Groq API timeout/error mid-demo	Every agent call wrapped in try/except degrading to a rule-based summary (already the pattern in notify.py) — rehearse the degraded path on purpose, don't just hope it's never needed
Judge asks 'how is confidence computed?'	Have the exact feature list (completeness, reading count, variance, method agreement) ready to state in one sentence — this was the #1 weakness found in the original audit, make it the #1 strength in the pitch
Judge asks 'is this leak location real sensor data?'	Answer directly: no, Level 1 hardware is out of scope for a 24-hour build by design, this is simulated to demonstrate what the system does once minimal instrumentation exists — matches the honesty already in the solution doc, don't oversell it live
Judge asks 'where did the town data come from?'	Lucknow's real published city-wide NRW% and revenue-loss figures, cited; zone-level breakdown is synthetic but mathematically consistent with that real aggregate — say this plainly, it's a strength not a weakness if stated upfront
Live demo network/DB failure	Deploy early (Hour 0-2), not at Hour 23 — a system that's been live for 20+ hours has already surfaced its integration bugs before judging starts

7.2 Suggested pitch flow

- Open with the real Lucknow numbers (55% NRW, █650M/year) — grounds the whole demo in something citable before touching the product.
- Show the Zone map with real data flowing from the new CSV pipeline, not a hardcoded fallback.
- Click into a flagged zone → open the AI Copilot → watch it investigate live, citing real water-balance/MNF/billing-anomaly evidence, ending in a structured recommendation with a real payback period.
- Show the billing-audit and leak-location screens that were previously invisible — this is the 'we had this the whole time, you just couldn't see it' moment.
- Approve the recommendation on screen → show the Revenue Recovered tracker actually move, closing the trace → tackle → convert loop the problem statement asks for.
- Close with the honest-limitations slide, carried over from the original solution doc — stating your own limits plainly is a credibility signal, not a weakness, in front of technical judges.

End of roadmap. This document should live in the new repo as docs/roadmap.pdf and DEPRECATIONS.md should be split out from Section 4 as its own file at the repo root.